SLAM & Attitude Estimation

How Robots and Headsets Know Where They Are

Roberto Marino, PhD

Computational Modeling for Robotics and Human Computer Interaction
University of Messina - FCRLab
roberto.marino@unime.it

May 19, 2026

UniMe
1548

What We Know So Far

In this course we have studied **serial manipulators**. A manipulator's world is simple: the base is fixed, every joint has an encoder, and the pose of the end-effector follows directly from $T = \mathrm{FK}(q)$.

The robot always knows exactly where it is. There is no estimation involved.

The key advantage of manipulators

Every degree of freedom is *measured directly* by an encoder. The world frame is anchored to the base. Geometry is enough — no probability, no filtering.

A New Kind of Problem

Now consider a **mobile robot** sweeping a warehouse, or a **VR headset** worn by someone moving freely.

There is no fixed base, no encoders on the joints of the world. The device must figure out its own position and orientation using only the sensors it carries.

This is fundamentally harder: the state is not measured — it must be *estimated* from noisy, imperfect data.

Two sub-problems

Attitude estimation
Which way am I pointing?
Recover orientation $R(t)$ from sensor data.

SLAM
Where am I â and what does the world look like?
Estimate position *and* build a map, simultaneously.

Why Not Just Use GPS?

GPS requires line-of-sight to satellites. It does not work indoors, in tunnels, or in warehouses with metal roofs.

Even outdoors, consumer GPS accuracy is **3–5 metres**. That is fine for a car on a road, but completely useless for a robot avoiding a 30 cm obstacle — or a headset that needs millimetre precision to avoid making the user sick.

Context	GPS?	Accuracy needed
Car navigation	✓	3–5 m
Warehouse robot	×	∼5 cm
Surgical robot	×	<1 mm
VR headset	×	<1 mm

The sensors used instead: IMU, LiDAR, cameras, wheel encoders.

How Do We Store "Which Way I Am Pointing"?

Before estimating orientation, we need a mathematical object to represent it. You already know three options from the course: rotation matrices, Euler angles, and quaternions.

They each make a different trade-off between *intuition*, *computational cost*, and *numerical stability*. Choosing the wrong one for the wrong task causes real problems.

Representation	Parameters
Rotation matrix $R \in \mathrm{SO}(3)$	9
Euler / RPY angles	3
Unit quaternion	4

All three encode only **3 degrees of freedom**. The differences lie in singularities, efficiency, and how well they behave inside a numerical filter.

Rotation Matrices

A rotation matrix $R \in \mathrm{SO}(3)$ satisfies $R^\top R = I$ and $\det R = +1$. Its columns are the body-frame axes expressed in the world frame — a very direct geometric meaning.

Rotation matrices are the right tool for **static calculations**: FK, IK, MDH transformations. They are *not* the right tool for continuous filtering. After many multiplications, rounding errors break the constraint $R^\top R = I$, and the matrix drifts away from being a valid rotation.

Elementary rotations (reminder)

$$R_z(\theta) = \begin{bmatrix} c_\theta & -s_\theta & 0 \\ s_\theta & c_\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$R = R_z(\psi)\, R_y(\theta)\, R_x(\phi)$$

Good for: FK, IK, MDH.
Bad for: continuous integration in a filter.

Euler Angles

Euler angles describe orientation as three successive rotations: roll (ϕ), pitch (θ), and yaw (ψ).

They are **intuitive**: a pilot immediately understands "the aircraft is banked 15Â° to the right and climbing at 5Â°." Three numbers is also compact.

The downside is a singularity called **gimbal lock** that appears when pitch reaches ± 90.

ZYX intrinsic convention

$$R = R_z(\psi)\, R_y(\theta)\, R_x(\phi)$$

Good for: displaying orientation to a human.
Bad for: anything that crosses $\theta = \pm 90$.

Gimbal Lock

When pitch $\theta = 90$, the matrix R loses one degree of freedom: roll and yaw both rotate around the same physical axis. Two separate knobs suddenly control the same motion.

Try it yourself: tilt your head back until you look straight up. Now "turn left" and "roll your head" feel identical. That is gimbal lock.

For a VR headset worn by someone who might look anywhere, or a drone doing a flip, this is fatal. A filter built on Euler angles crashes at this configuration.

At $\theta = 90$:

$$R = \begin{bmatrix} 0 & 0 & 1 \\ s_{\phi-\psi} & c_{\phi-\psi} & 0 \\ -c_{\phi-\psi} & s_{\phi-\psi} & 0 \end{bmatrix}$$

Only the *difference* $\phi - \psi$ is identifiable. Roll and yaw are indistinguishable.

The rule: use Euler angles only for human display, never inside a filter or integrator.

Unit Quaternions

Unit quaternions solve the gimbal lock problem entirely. A rotation of angle α about axis $\hat{n}$ is encoded as:

$$\boldsymbol{q} = \left[\cos\tfrac{\alpha}{2},\ \hat{n}\sin\tfrac{\alpha}{2}\right], \qquad \|\boldsymbol{q}\| = 1.$$

Every orientation has a valid quaternion, at every angle, without exception.

Integrating a gyroscope reading is a simple multiplication followed by a re-normalisation $\boldsymbol{q} \leftarrow \boldsymbol{q}/\|\boldsymbol{q}\|$. Much cheaper and more stable than re-orthogonalising a 3×3 matrix.

The practical rule

Store and filter in quaternions.
Display to the user in Euler angles.
Interpolate between two orientations with SLERP.

Every drone autopilot, every IMU filter, and every VR runtime follows this rule. The functions `quat2R()`, `R2quat()`, and `slerp()` in your library implement the conversions.

SLERP: Smooth Orientation Interpolation

Given two orientations $\boldsymbol{q}_0$ and $\boldsymbol{q}_1$, we often need a smooth intermediate pose — for example, between two IMU samples during rendering.

Simple linear interpolation $(1-t)\boldsymbol{q}_0 + t\boldsymbol{q}_1$ is wrong: it leaves the unit sphere, giving a non-unit quaternion and a non-constant angular velocity.

SLERP (Spherical Linear Interpolation) is the geodesic arc on S^3: it stays on the sphere, moves at constant speed, and takes the shortest path.

SLERP formula

$$\mathrm{Slerp}(\boldsymbol{q}_0, \boldsymbol{q}_1, t) = \frac{\sin(1-t)\Omega}{\sin\Omega}\,\boldsymbol{q}_0 + \frac{\sin t\Omega}{\sin\Omega}\,\boldsymbol{q}_1$$

$$\Omega = \arccos(\boldsymbol{q}_0 \cdot \boldsymbol{q}_1)$$

Ensure $\boldsymbol{q}_0 \cdot \boldsymbol{q}_1 > 0$ (shortest arc).

Used in: `ctraj()` in your library, VR frame rendering.

The Accelerometer

The **accelerometer** measures the force the chip feels, minus gravity.

When the device is *still*, only gravity acts on it. The reading tells us which way "down" is — from which we can recover roll and pitch.

When the device is *moving*, inertial forces mix with gravity and the measurement becomes contaminated. **Yaw** (rotation around the vertical axis) is always unobservable: gravity is vertical and carries no horizontal information.

Accelerometer model

$$\tilde{\boldsymbol{a}} = R^{\top}(\boldsymbol{a}_{\text{body}} - \boldsymbol{g}) + \boldsymbol{b}_a + \boldsymbol{\eta}_a$$

When still, $\boldsymbol{a}_{\text{body}} \approx \mathbf{0}$:

$$\phi = \operatorname{atan2}(\tilde{a}_y,\, \tilde{a}_z)$$

$$\theta = \operatorname{atan2}\Big(-\tilde{a}_x,\, \sqrt{\tilde{a}_y^2 + \tilde{a}_z^2}\Big)$$

The Gyroscope and the Drift Problem

The **gyroscope** measures angular velocity. Integrating it over time gives orientation.

Every gyroscope has a tiny constant *bias*: a small offset in its reading that never goes away. Integrating that offset accumulates error without stopping.

Analogy: walking blindfolded. Each step is slightly off, and over 100 steps you end up noticeably far from where you think you are.

Drift quantification

Bias of $0.1/\mathrm{s}$ (typical consumer MEMS):

After 10 s	1 error
After 60 s	6 error
After 10 min	60 error

The gyro is accurate *short-term*; the accelerometer is stable *long-term*. They are complementary.

Fusing the Two Sensors

Because the two sensors have *opposite* weaknesses, fusing them gives something neither can provide alone: an estimate that is both fast and drift-free.

The gyroscope dominates in the short term. The accelerometer gently corrects drift over time. A third sensor, the **magnetometer**, adds an absolute yaw reference — but it is easily disrupted by metal structures and motors, so many systems ignore it indoors.

Sensor	Good at	Bad at
Gyro	Fast rotation	Long-term drift
Accel	Gravity / tilt	Fast motion, no yaw
Mag	Absolute yaw	Interference

Goal: estimate $R(t)$ from $\{\tilde{\boldsymbol{\omega}}, \tilde{\boldsymbol{a}}, \tilde{\boldsymbol{m}}\}$. This is the *attitude estimation* problem.

The Complementary Filter

The simplest fusion strategy translates the blending idea directly into code.

Step 1 — propagate with the gyroscope:
$\boldsymbol{q}_\omega = \boldsymbol{q}_k \otimes \exp(\tilde{\boldsymbol{\omega}}\, \Delta t/2)$.
Step 2 — extract a tilt estimate $\boldsymbol{q}_a$ from the accelerometer.
Step 3 — blend the two with SLERP: $\boldsymbol{q}_{k+1} = \mathrm{Slerp}(\boldsymbol{q}_\omega, \boldsymbol{q}_a, \alpha)$.

With $\alpha = 0.02$: 98% gyroscope, 2% accelerometer correction.

One tuning parameter (α). No matrices. No inversions. Runs on the simplest microcontroller.

Main limitation: the accelerometer correction assumes the device is not accelerating. This fails when the robot moves dynamically — a running drone, a cornering car, an active VR user.

For slow, stable platforms it works beautifully.

The Kalman Filter — the Optimal Blend

The **Extended Kalman Filter (EKF)** is the rigorous version. Instead of a fixed α, it computes the *optimal weight* at every step based on how uncertain each source actually is.

It also estimates the **gyro bias** as part of the state. As the filter runs, it learns how much the gyroscope is lying and subtracts that bias before integrating — fixing drift at its root.

State vector

$$\boldsymbol{x} = \begin{bmatrix} \boldsymbol{q} \\ \boldsymbol{b}_g \end{bmatrix} \in \mathbb{R}^7$$

Orientation $\boldsymbol{q}$ (4 numbers) + gyro bias $\boldsymbol{b}_g$ (3 numbers).

Predict and Update

The EKF alternates between two steps at every timestep.

Predict (gyroscope, runs fast): use the motion model to move the estimate forward in time, and grow the uncertainty accordingly.

Update (accelerometer correction): compare what the sensor reads with what the model predicted it should read. Use the mismatch to correct the estimate. The **Kalman gain** K decides how strongly to apply the correction — balancing trust between model and sensor based on their covariances.

The cycle

Predict:

$$\hat{\boldsymbol{x}}^- = f(\hat{\boldsymbol{x}}, \tilde{\boldsymbol{\omega}}), \quad P^- = FPF^\top + Q$$

Update:

$$K = P^- H^\top (HP^- H^\top + R)^{-1}$$

$$\hat{\boldsymbol{x}} \mathrel{+}= K(\tilde{\boldsymbol{a}} - h(\hat{\boldsymbol{x}}^-))$$

Which Filter Should You Use?

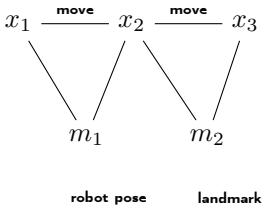
	Complementary	**Madgwick**	**Kalman (EKF)**
Complexity	Very simple	Simple	Medium
Estimates gyro bias	No	Partly	Yes ✓
Works during motion	Poorly	Well	Well ✓
Tuning	1 number	1 number	Noise matrices
Used in	DIY drones	VR headsets	Professional robots

Madgwick is a clever middle ground: it achieves EKF-quality results with a single tuning parameter, which is why Meta Quest and HoloLens use it.

The Blindfolded Explorer

Imagine exploring an unknown room with a flashlight while blindfolded. You move around counting steps and tracking every turn. Occasionally your flashlight illuminates something recognisable — a chair, a door, a painting. Each time, you update your mental map and correct where you think you are.

SLAM is exactly this. The robot is the blindfolded explorer; its sensors are the flashlight; environmental features are the furniture.



The Chicken-and-Egg Problem

SLAM has a fundamental paradox at its heart.

To **localise** accurately, you need a good map.
To build a **good map**, you need to know where you are.

Neither can be solved first. Both must be estimated *jointly* and *simultaneously* — which is what makes SLAM both difficult and fascinating.

The two sources of error.

Dead reckoning drift: integrating motion commands accumulates position error that grows without bound.

Observation noise: seeing a landmark gives only a noisy, uncertain measurement of its position.

Both must be modelled probabilistically.

Dead Reckoning

The simplest tracking strategy is to **integrate every motion** from a known starting point. For a wheeled robot this means reading the wheel encoders; for an IMU-only system it means double-integrating the accelerometer.

It works for a while. But small errors compound. A heading error of just 0.1 causes $1.7\,\mathrm{m}$ of lateral drift per kilometre of travel. A $1\,\mathrm{mg}$ accelerometer bias drifts $18\,\mathrm{m}$ in one minute.

Think of sailors before GPS. They kept a running estimate of position using speed and heading. When they could see a star or a lighthouse, they corrected. Without those corrections they drifted hundreds of miles off course.

A robot does the same: dead reckoning moves the estimate forward; **landmark observations correct it**.

EKF-SLAM

EKF-SLAM extends the Kalman filter to handle both robot pose and landmark positions simultaneously. The state is one big vector:

$$\boldsymbol{X} = \big[\underbrace{x,\ y,\ \psi}_{\text{robot}},\ \underbrace{\ell_x^1,\ \ell_y^1,\ \ldots}_{\text{landmarks}}\big].$$

The same predict–update rhythm applies. **Predict:** move the robot pose using odometry; landmarks stay fixed. **Update:** when a landmark is seen, correct both the robot pose and all correlated landmark estimates at once.

A beautiful side effect. When the robot gets a precise fix from one landmark, the improvement propagates to *all* landmarks — even those not currently visible — because the filter tracks how everything is correlated.

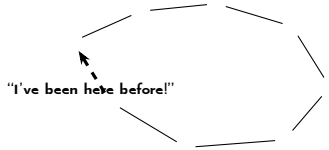
The cost. Updating the full state takes time proportional to n^2, where n is the number of landmarks. For large maps, faster alternatives (particle filters, graph SLAM) are needed.

Loop Closure

Even with frequent landmark updates, drift accumulates on long journeys.

The most powerful single correction is **loop closure**: the robot returns to a place it has visited before and *recognises* it. The mismatch between where it *thinks* it is and where it *knows* it must be corrects all the drift from the entire loop at once.

In VR, this happens every time you leave and re-enter a room: the headset matches the current view to a stored map and instantly recovers its position.



LiDAR vs. Cameras

Early SLAM systems used **LiDAR**: a spinning laser that fires thousands of pulses per second and measures distance directly in 3-D. Accurate, unaffected by lighting — but heavy, expensive, and power-hungry. Not an option for a headset you wear on your head.

Cameras are the natural replacement. A small camera module weighs a few grams, costs a few dollars, and sips milliwatts. The challenge: a camera does not measure depth directly. With two cameras at a known separation (*stereo*), depth can be recovered from the disparity between the two images.

	LiDAR	Camera
Direct depth	Yes	No (stereo helps)
Cost	High	Low
Weight	Heavy	Light
Light-sensitive	No	Yes
Used in	Outdoor robots	Headsets, phones

Visual-Inertial Odometry (VIO)

Combining cameras with an IMU gives **Visual-Inertial Odometry**. The principle is the same as fusing gyroscope and accelerometer for attitude — but now the camera plays the role of the absolute reference.

The IMU at 1000 Hz tracks fast rotations accurately but drifts. The camera at 60 Hz detects visual features in the environment and provides an absolute anchor for the position. A filter fuses both, and the result is a full 6-DoF pose that is both fast and drift-free.

The key idea

Camera features are the "landmarks" of SLAM.

Corners of furniture, texture patches, door frames — the camera tracks them across frames and uses their consistency to correct the IMU's accumulated drift.

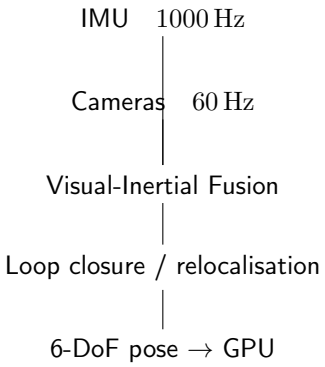
Same math as EKF-SLAM; different sensor.

Inside-Out Tracking

Modern VR/AR headsets use **inside-out** tracking: every sensor is on the device itself. No external base stations, no room setup required.

The Meta Quest 3 has four wide-angle cameras and an IMU. A VIO algorithm runs entirely on-device, producing a 6-DoF head pose at 72–120 Hz with end-to-end latency below 20 ms.

Exceeding that latency threshold makes the image lag behind head movement — causing the *vergence-accommodation conflict* that triggers motion sickness. Tracking is, quite literally, a problem of milliseconds.



Summary

Problem	Sensors	Algorithm	Example
Attitude only	IMU	Complementary / Madgwick	Drone stabilisation
Attitude + yaw	IMU + magnetometer	EKF	Phone compass
SLAM (indoor)	Camera + IMU	VIO + loop closure	Meta Quest, ARKit
SLAM (outdoor)	LiDAR + IMU	LIO-SAM, Cartographer	Autonomous car

The unifying idea across every row:
a **predict–update loop** that propagates an uncertain belief forward (using a motion model) and corrects it with sensor observations.

Building a robot that knows where it is is, at its core, a problem of **probabilistic inference** — not just geometry.